

Wykład 13

LoRA-Low Rank Adaptation

Dr Urszula Grzybowska

LLM-jak działają?



Dane

GPT-3, jeden z największych LLM-ów jest trenowany na 570 GB danych tekstowych. Mniejsze LLM-y mogą potrzebować mniej - może 10-20 GB lub nawet 1 GB gigabajtów - ale to i tak dużo. Tekst czyścimy i dzielimy na pnie- tokeny

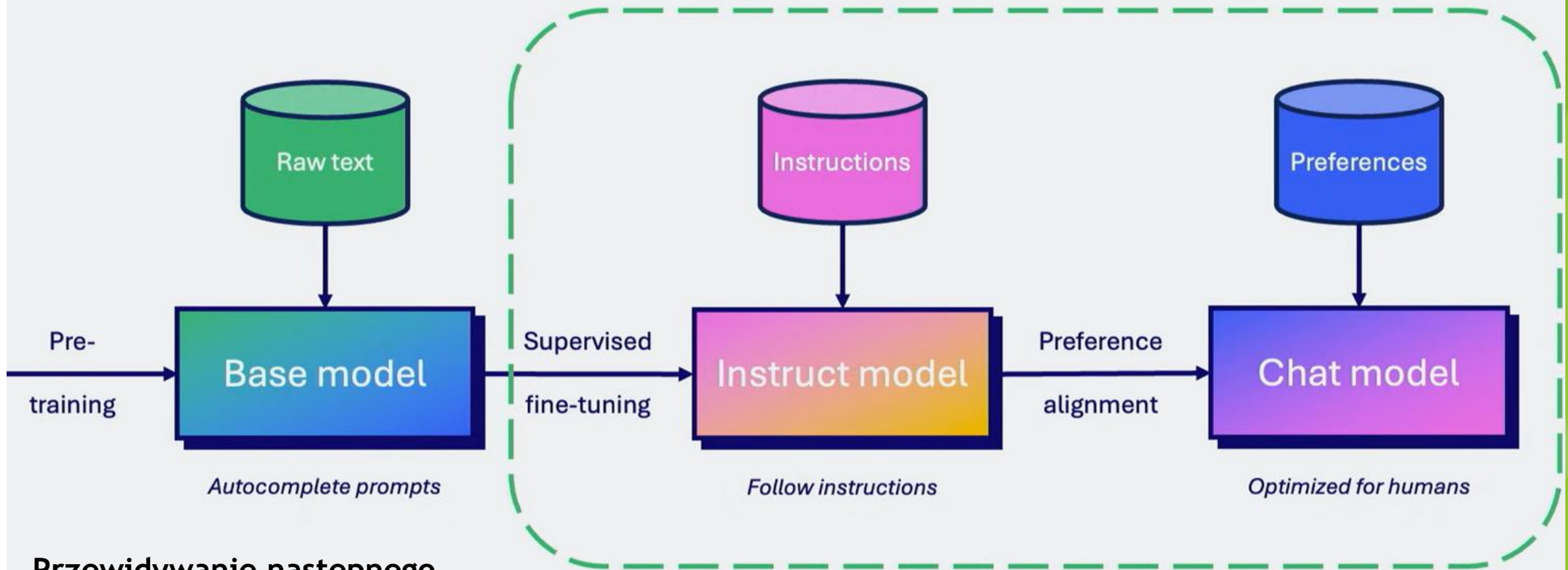
Transformery
Mechanizm uwagi

Model
Np. GPT
GPT3 ma
175miliarów
parametrów

Cel: dla danego ciągu tokenów
przewidywanie następnego tokenu

- Model bazowy przewiduje następny token.
- Nie jest w stanie odpowiedzieć na pytania, itp.

Post-Training



Przewidywanie następnego tokenu- next token prediction

Uczymy model interpretowania instrukcji i odpowiadania na pytania

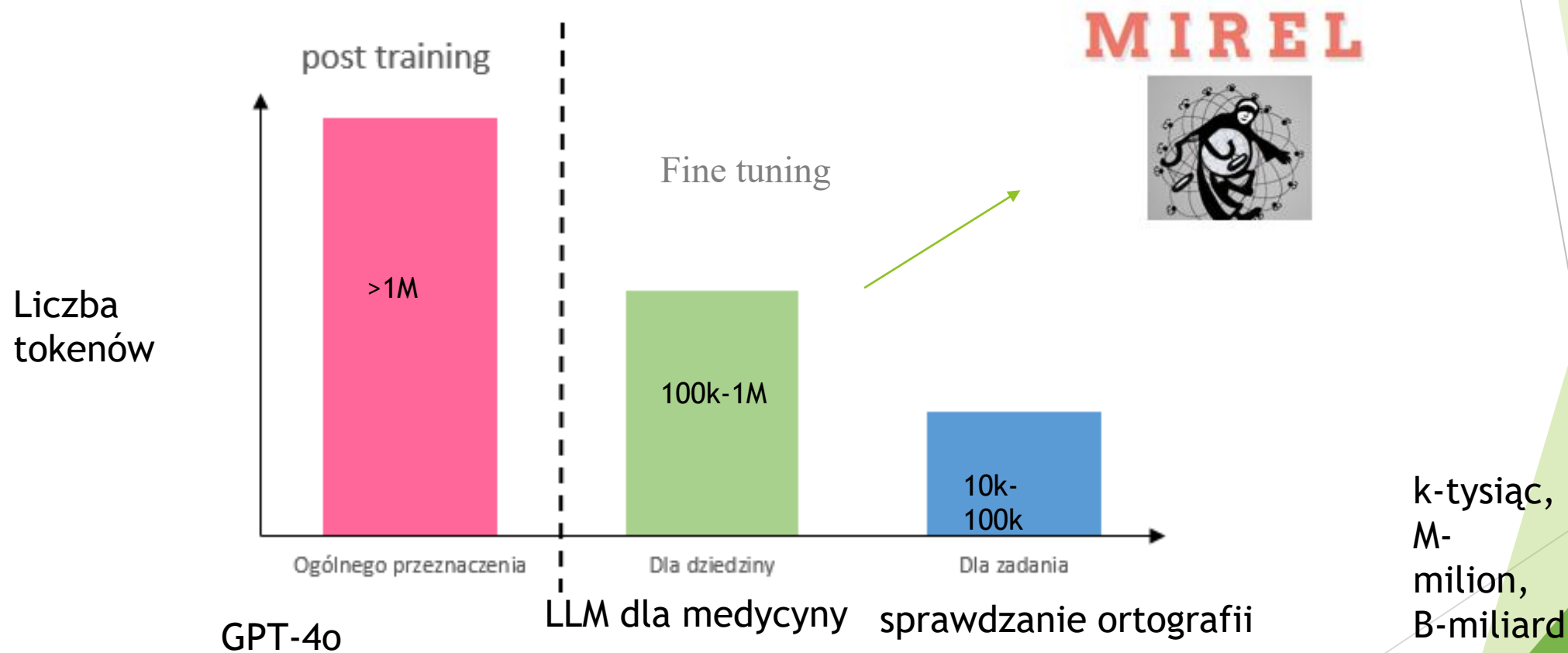
Uczymy model prawidłowych odpowiedzi

Fine-tuning

Dostrajanie (fine-tuning) modeli językowych LLM, to proces polegający na wykorzystaniu wstępnie wytrenowanego dużego modelu językowego (LLM) np. GPT, Llama i poddaniu go dalszemu szkoleniu na bardziej szczegółowym zbiorze danych, tak aby osiągał lepsze wyniki w zakresie konkretnego zadania, stylu lub dziedziny.

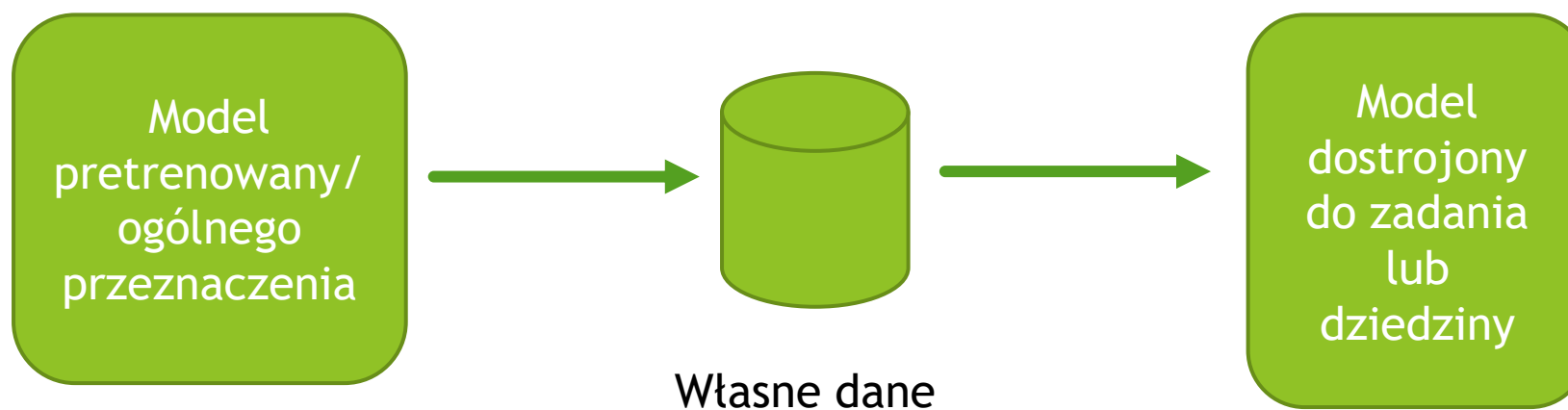
Dostrajanie modelu LLM = dostosowywanie ogólnego modelu sztucznej inteligencji poprzez dalsze szkolenie go na konkretnych danych, tak aby osiągał lepsze wyniki w określonych zadaniach.

Post training-przekształcenie modelu bazowego w użyteczną aplikację



**Generative Pre-trained Transformer
4 Omni (GPT-4o)**

Fine-tuning



Llama-3.2-1B 1.2B

Llama-3-8B 8.0B

Mistral-7B 7.2B

Llama-3-70B 70B

GPT

Llama

Claude

Polecam

MIT 6.S191 (Liquid AI): Large Language Models

https://www.youtube.com/watch?v=_HfdncCbMOE&t=487s

Wykład pojawił się 21 kwietnia 2025 r. Nie ma jeszcze z 2026 roku.

Wykładowca: **Maxime Labonne** (Liquid AI)

<https://github.com/mlabonne/llm-course>



TRL - Transformer Reinforcement Learning

- 00:01 - Introduction to LLM Post-Training
- 01:47 - Preference Alignment and Data Set Considerations
- 03:28 - When and Why to Fine-Tune: Use Cases and Data Quality
- 10:28 - Data Complexity, Formats, and Generation Pipelines
- 15:47 - Preference Data Generation and Chat Templates
- 19:11 - Fine-Tuning Libraries and Techniques
- 24:30 - Training Parameters, Experiment Monitoring, and Model Merging
- 33:42 - Model Merging Case Study and Evaluation Challenges
- 41:10 - Evaluation Methods and Future Trends: Test-Time Compute Scaling

Wagi modelu

Podczas uczenia wagi modelu są dostosowywane w celu zminimalizowania straty.
W architekturach transformerów mamy:

- Wagi osadzania: mapują tokeny na wektory (i odwrotnie)
- Wagi uwagi: projekcje Q, K, V oraz projekcja wyjściowa
- Wagi sieci FFN: macierze projekcji w górę i w dół
- Wagi warstwy normalizacji (Layer normalization)

Każdy parametr jest liczbą zmiennoprzecinkową.
Przy precyzji FP16 każdy parametr zajmuje 2 bajty.
(half-precision floating point)

Pamięć modelu (w bajtach) = Liczba parametrów × Liczba bajtów na parametr
8 miliardów × 2 bajty = 16 GB

<https://agentfactory.panaversity.org/docs/Turing-LLMOps-Proprietary-Intelligence/llm-architecture-compute/parameter-counting>

<https://llmdb.com/models>

 107 Models  14 Providers

Explore The LLM Database

Compare 107+ language models with detailed specifications and benchmarks

 42 Open Weights

 65 Proprietary

 Latest: Claude Opus 4.6

Po co fine-tuning

Dostrajanie przebiega na małym zbiorze danych w porównaniu z danymi do uczenia modelu (tysiące kontra miliardy tokenów).

W ramach dostrajania dokonujemy precyzyjnych korekt, a nie szkolimy model od podstaw.

LoRA sprawdza się w tej funkcji dobrze.

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
Microsoft Corporation

2021 r.

1 INTRODUCTION

Many applications in natural language processing rely on adapting *one* large-scale, pre-trained language model to *multiple* downstream applications. Such adaptation is usually done via *fine-tuning*, which updates all the parameters of the pre-trained model. The major downside of fine-tuning is that the new model contains as many parameters as in the original model. As larger models are trained

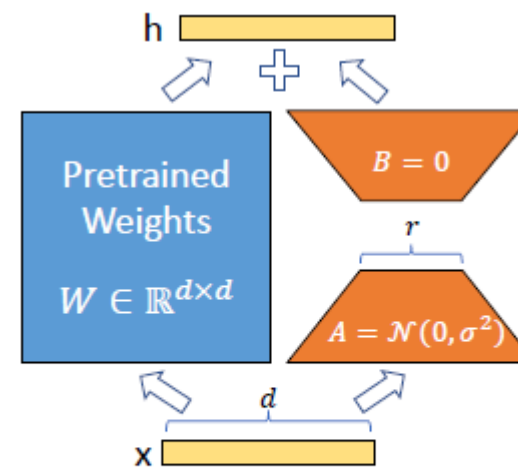


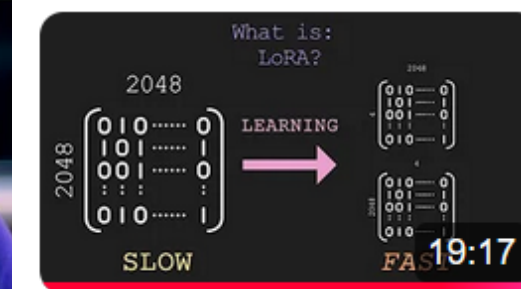
Figure 1: Our reparametrization. We only train A and B .



Low-rank Adaption of Large Language Models Part 2: ...

Chris Alexiuk

27 tys. wyświetleń • 3 lata temu



Low-rank Adaption of Large Language Models: Explaining ...

Chris Alexiuk

133 tys. wyświetleń • 3 lata temu

Z dubbingiem automatycznym

What is Low-Rank Adaptation (LoRA) | explained by the inventor



Edward Hu

9,45 tys. subskrybentów

Subskrybuj

Praktyczne uwagi

LoRA: Low-Rank Adaptation of Large Language Models - Explained visually + PyTorch code from scratch



Umar Jamil

83 tys. subskrybentów

Subskrybuj



1,8 tys.

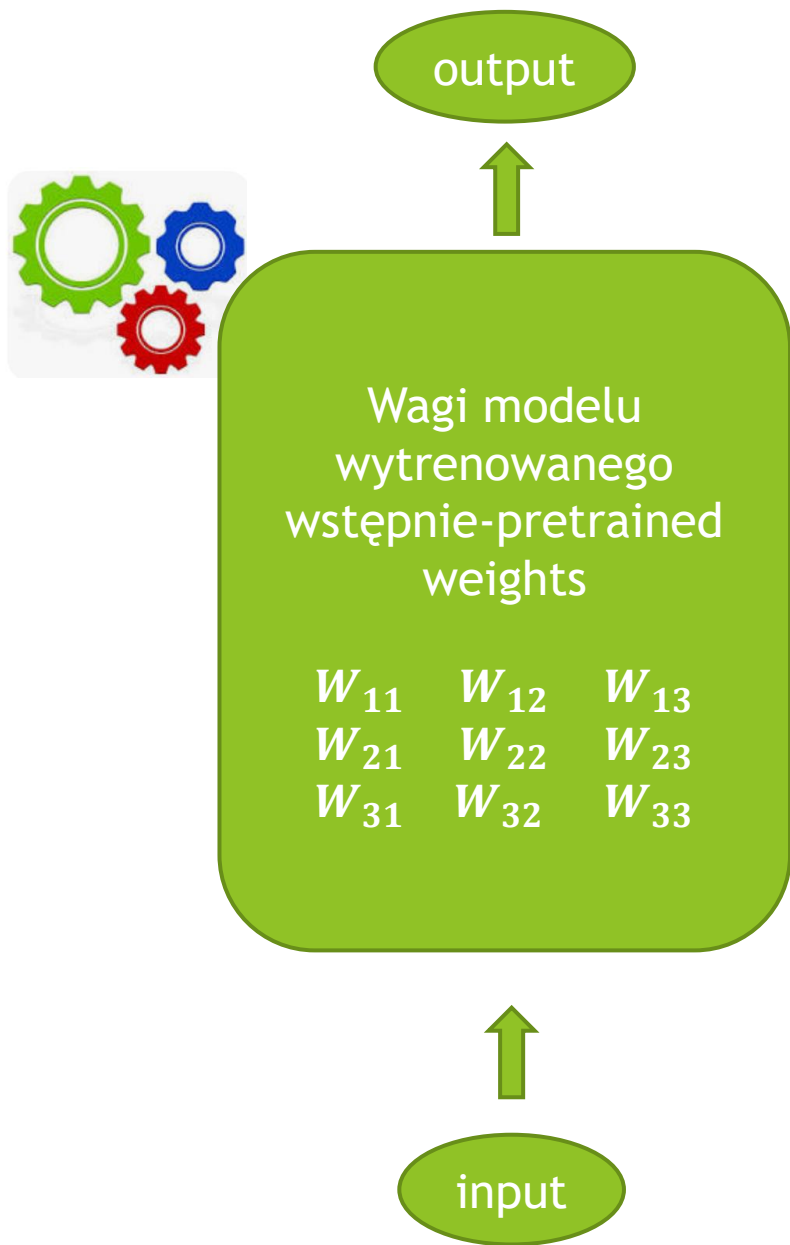


Udostępnij



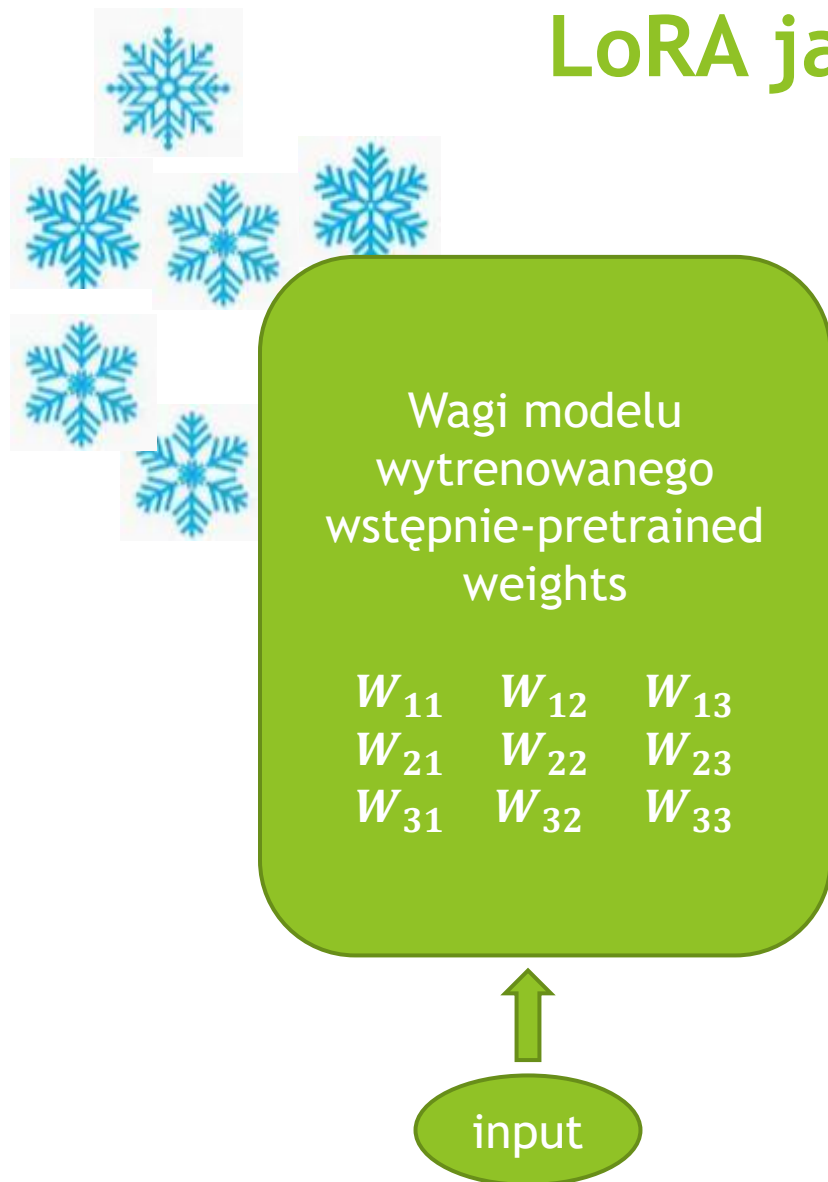
Zapisz





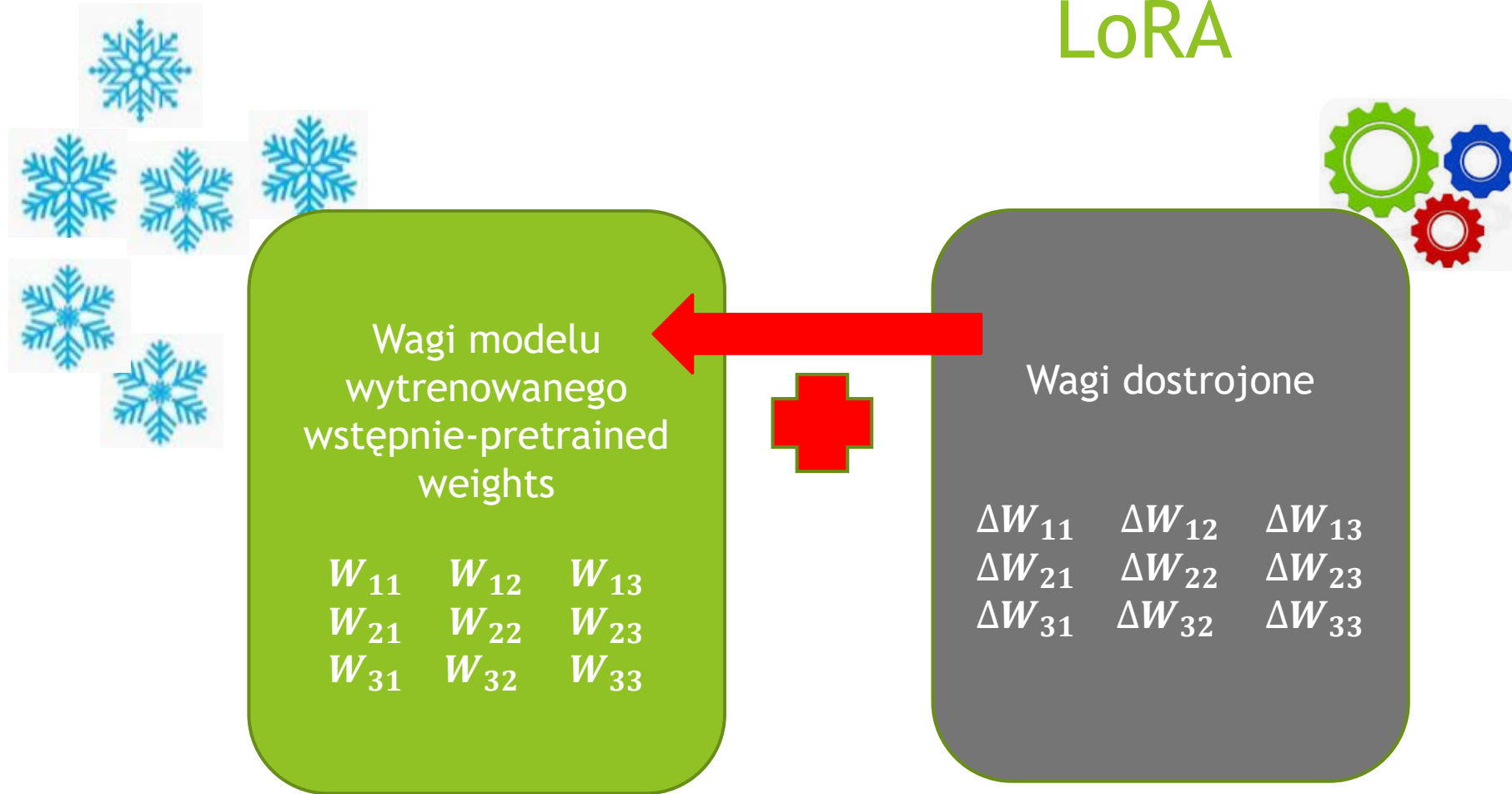
Model	Miliardy Parameters	Layers	Hidden	Heads	FFN	VRAM (FP16)
Llama-3.2-1B	1.2B	16	2,048	32	8,192	2.4 GB
Llama-3-8B	8.0B	32	4,096	32	14,336	16 GB
Mistral-7B	7.2B	32	4,096	32	14,336	14.4 GB
Llama-3-70B	70B	80	8,192	64	28,672	140 GB

LoRA jako metod PEFT



Metoda PEFT (Parameter-Efficient Fine-Tuning) polega na dostosowywaniu dużych, wstępnie wytrenowanych modeli językowych (LLM) do konkretnych zadań poprzez **aktualizację jedynie niewielkiego podzbioru ich parametrów**, przy czym większość parametrów modelu pozostaje niezmienną. Takie podejście zazwyczaj pozwala zmniejszyć nakłady obliczeniowe i skrócić czas uczenia modeli LLM pod kątem zadań specjalistycznych bez konieczności ponownego uczenia całego modelu.

LoRA



LoRA



Wagi modelu
wytrenowanego
wstępnie-pretrained
weights

$$W = \begin{pmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \end{pmatrix}$$

Wagi dostrojone

$$\Delta W = \begin{pmatrix} \Delta W_{11} & \Delta W_{12} & \Delta W_{13} \\ \Delta W_{21} & \Delta W_{22} & \Delta W_{23} \\ \Delta W_{31} & \Delta W_{32} & \Delta W_{33} \end{pmatrix}$$



Rząd macierzy- rank of a matrix

Minor rzędu k.

Minorem stopnia **k** macierzy

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

nazywamy macierz kwadratową otrzymaną z macierzy A przez skreślenie **n-k** kolumn i **m-k** wierszy.

Minorem bazowym macierzy A nazywamy minor największego stopnia o wyznaczniku różnym od zera.

Rzędem macierzy nazywamy **stopień minora bazowego**.

Rząd macierzy- rank of a matrix

$$r \begin{bmatrix} 1 & 0.5 & 2 \\ 1 & 0.5 & -1 \\ 2 & 1 & 3 \end{bmatrix} = 2$$

x2

Rank(A)=r(A)=Liczba liniowo niezależnych wektorów (kolumn lub wierszy).

W mnożeniu macierzy A i B mnożymy wiersze A przez kolumny B

$$\begin{pmatrix} \boxed{1} & \boxed{-1} \\ \boxed{2} & \boxed{3} \\ -1 & 2 \end{pmatrix} \cdot \begin{pmatrix} \boxed{2} & 3 \\ \boxed{1} & -1 \end{pmatrix} = \begin{pmatrix} \boxed{1} & 4 \\ 7 & 3 \\ 0 & -5 \end{pmatrix}$$

$$m \times r \cdot r \times p = m \times p$$

$$d \times 1 \cdot 1 \times k = d \times k$$

9 parametrów

$$\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} (2 \quad 1 \quad 3) = \begin{pmatrix} 2 & 1 & 3 \\ 4 & 2 & 6 \\ 2 & 1 & 3 \end{pmatrix}$$

6 parametrów

LoRA

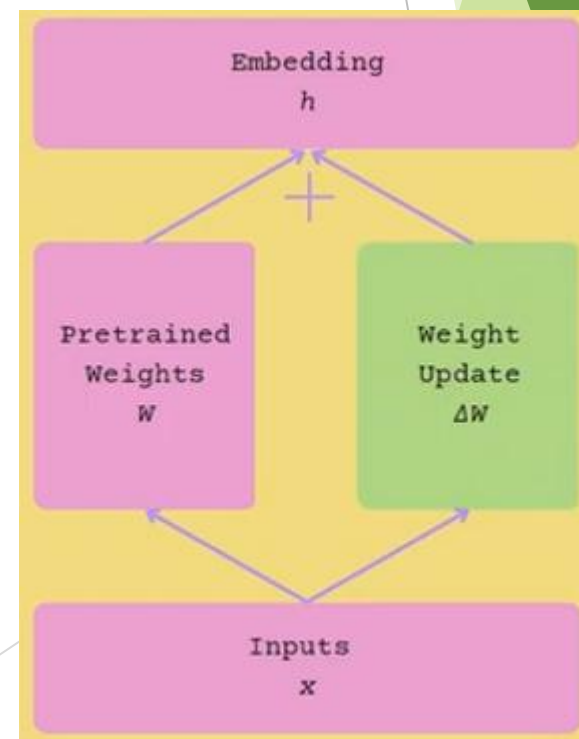
$W \in R^{d \times k}$ macierz preternowanych wag (warstwy modelu, macierzy zapytania, klucza, itd.)

$$W + \Delta W = W + B \cdot A \quad B \in R^{d \times r}, \quad A \in R^{r \times d}$$

$$\Delta W = B \cdot A, \quad r \ll \min(d, k)\text{-hiperparametr}$$

W oraz ΔW są mnożone przez ten sam wektor x

$$h = Wx + BAx$$



https://huggingface.co/docs/peft/developer_guides/lora

LoRA

Copy page

LoRA is low-rank decomposition method to reduce the number of trainable parameters which speeds up finetuning large models and uses less memory. In PEFT, using LoRA is as easy as setting up a LoraConfig and wrapping it with get_peft_model() to create a trainable PeftModel.

This guide explores in more detail other options and features for using LoRA.

Creates Low Rank Adapter (LoRA) model from a pretrained transformers model.

The method is described in detail in <https://huggingface.co/papers/2106.09685>.

Example:

```
>>> from transformers import AutoModelForSeq2SeqLM
>>> from peft import LoraModel, LoraConfig

>>> config = LoraConfig(
...     task_type="SEQ_2_SEQ_LM",
...     r=8,
...     lora_alpha=32,
...     target_modules=["q", "v"],
...     lora_dropout=0.01,
... )

>>> model = AutoModelForSeq2SeqLM.from_pretrained("t5-base")
>>> lora_model = LoraModel(model, config, "default")
```

Simple LoRA Implementation.ipynb

```
from peft import LoraConfig, get_peft_model
```

```
config = LoraConfig(  
    r=8,  
    lora_alpha=16,  
    target_modules=["query_key_value"],  
    lora_dropout=0.05,  
    bias="none",  
    task_type="CAUSAL_LM"  
)
```

```
model = get_peft_model(model, config)  
print_trainable_parameters(model)
```



Low-rank Adaption of Large Language Models Part 2: ...

Chris Alexiuk

27 tys. wyświetleń • 3 lata temu

TaskType	Meaning
CAUSAL_LM	Causal language modeling (GPT-style, next-token prediction)
SEQ_2_SEQ_LM	Sequence-to-sequence (e.g., T5, BART)
SEQ_CLS	Sequence classification
TOKEN_CLS	Token classification (e.g., NER)
QUESTION_ANS	Question answering
FEATURE_EXTRACTION	Embedding models

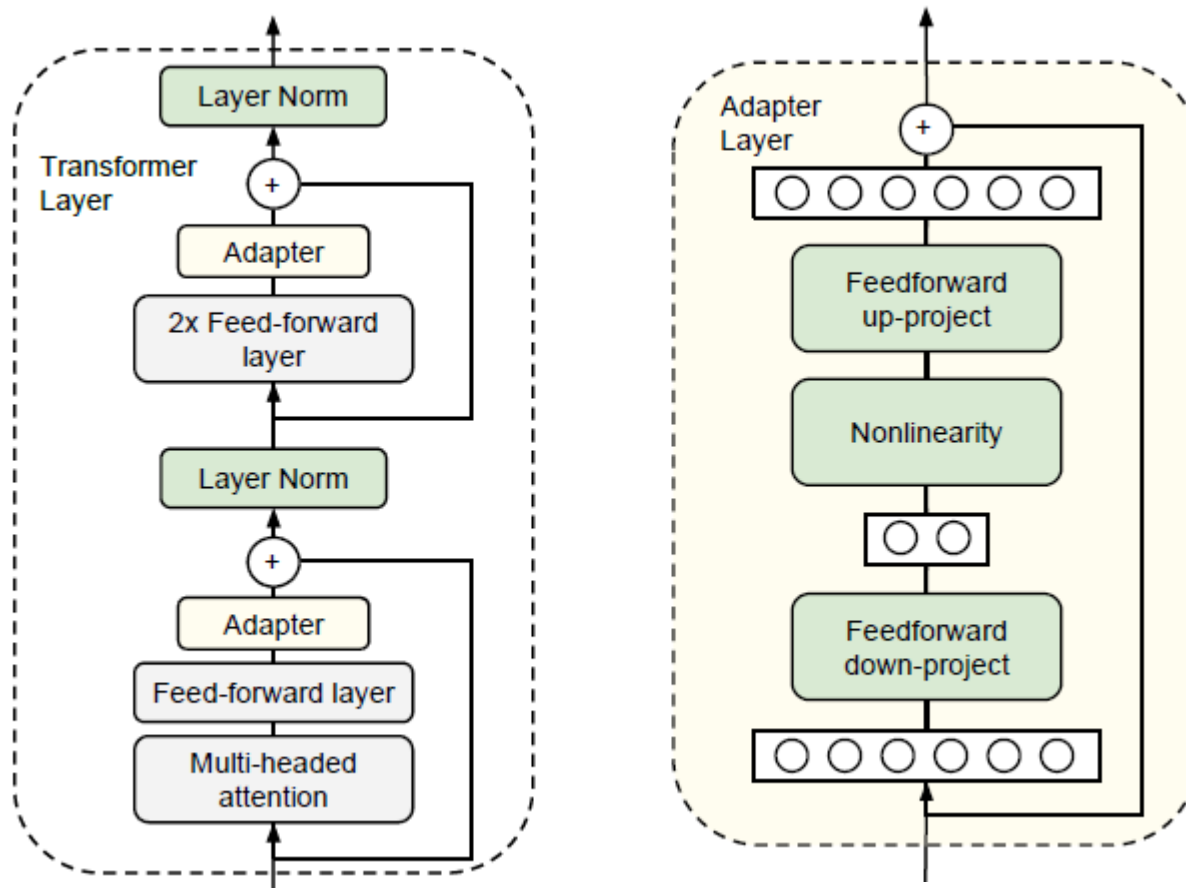
Korzyści ze stosowania LoRA

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Dokładność testowa w modelach dla WikiSQL i MultiNLI po zastosowaniu LoRA do różnych typów wag (macierze W) w GPT-3, przy tej samej liczbie parametrów podlegających uczeniu. Dostosowanie zarówno W_q , jak i W_v zapewnia ogólnie najlepszą wydajność.

- Dużo mniej parametrów do uczenia. Jeżeli $d = 1000, k = 5000, dxk = 5000000$; Dla $r=5$ mamy $(dxr) + (rxk) = 5000 + 25000 = 30000$. Jest to mniej niż 1% oryginalnej liczby parametrów.
- Mniej parametrów = mniejsze zapotrzebowanie na miejsce na dysku

Inne techniki-Adaptery



Parameter-Efficient Transfer Learning for NLP

Neil Houlsby¹ Andrei Giurgiu^{1*} Stanislaw Jastrzebski^{2*} Bruna Morrone¹ Quentin de Laroussilhe¹
Andrea Gesmundo¹ Mona Attariyan¹ Sylvain Gelly¹

Inne techniki-prefix tuning

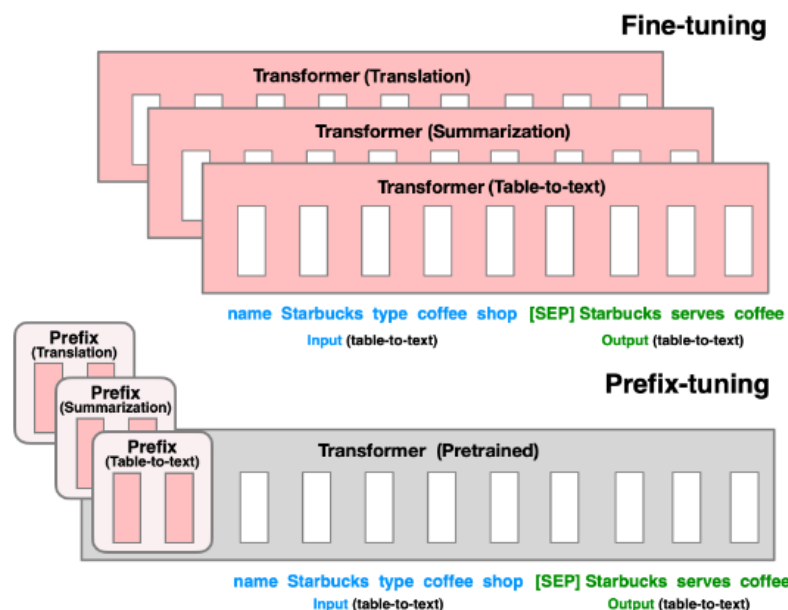


Figure 1: Fine-tuning (top) updates all Transformer parameters (the red Transformer box) and requires storing a full model copy for each task. We propose prefix-tuning (bottom), which freezes the Transformer parameters and only optimizes the prefix (the red prefix blocks). Consequently, we only need to store the prefix for each task, making prefix-tuning modular and space-efficient. Note that each vertical block denote transformer activations at one time step.

Prefix-Tuning: Optimizing Continuous Prompts for Generation

Xiang Lisa Li

Stanford University

xlisali@stanford.edu

Percy Liang

Stanford University

pliang@cs.stanford.edu

LoRA & QLoRA Fine-tuning Explained In-Depth

 Z dubbingiem automatycznym



Mark Hennings

10,1 tys. subskrybentów

Subskrybuj

 5,1 tys.

DoRA: Faster than LoRA for Fine-Tuning LLMs



Super Data Science: ML & AI Podcast with Jon Krohn

247 tys. subskrybentów

Subskrybuj